



UFOP

Trabalho Prático: Refatoração, Testes e DevOps

CSI410 – Engenharia de Software II

Luccas Vinicius – 20.1.8015 Davi Zanoti – 24.1.8062 (Fork:

github.com/luccas00/Car-Rental-App)

19 de julho de 2026

icea
Instituto de Ciências
Exatas e Aplicadas

DECSI
DEPARTAMENTO DE
COMPUTAÇÃO E SISTEMAS



Resumo Executivo

O

Este trabalho aplicou práticas de Engenharia de Software em um projeto real de código aberto: **Car Rental App**.

O foco principal foi a resolução da **Issue #216**, relacionada à correção do uso de injeção de dependência na tela de login.

Também foram entregues:

- documentação de arquitetura;
- análise de padrões, code smells e SOLID;
- pipeline GitHub Actions;
- testes automatizados adicionais;
- fluxo de trabalho com branches e Pull Requests.



UFOP

Sumário

1 Contexto

► Contexto

► Fluxo de Desenvolvimento

► Arquitetura

► Refatoração

► Qualidade

► Testes e DevOps

► Resultados



UFOP

Projeto, Dupla e Escopo

1 Contexto

Projeto escolhido

Car Rental App

Matrix-Developers/Car-Rental-App

Sistema

Aplicação desktop em C# / Windows Forms para gestão de locadora de veículos.

Integrantes

- **Luccas Vinicius** – 20.1.8015;
- **Davi Zanoti** – 24.1.8062.

Entrega

Fork, PRs, issue real, documentação, testes e CI/CD.



UFOP

Issue Selecionada: #216

1 Contexto

Problema original

A TelaLogin inicializava manualmente o DbContext, repositórios e serviços de aplicação, mesmo o projeto já utilizando Autofac.

Impacto técnico

A tela de login ficou acoplada à infraestrutura e assumiu responsabilidade de composição de dependências.

Entrega principal

Refatorar o fluxo de autenticação para que a criação de dependências fique centralizada no container Autofac.



UFOP

Sumário

2 Fluxo de Desenvolvimento

▶ Contexto

▶ Fluxo de Desenvolvimento

▶ Arquitetura

▶ Refatoração

▶ Qualidade

▶ Testes e DevOps

▶ Resultados



UFOP

Fluxo de Trabalho no Git

2 Fluxo de Desenvolvimento

Estratégia

- `main`: versão final do fork;
- `DEV`: branch de integração;
- branches específicas por entrega;
- integração via Pull Requests.

Fluxo simplificado

<code>refactor/...</code>	Issue #216
<code>ci/...</code>	GitHub Actions
<code>docs/...</code>	Documentação
<code>test/...</code>	Testes de DI
<code>DEV -> main</code>	Release final



UFOP

Pull Requests Criados

2 Fluxo de Desenvolvimento

PR	Tema	Objetivo
#1	Refatoração	Resolver dependências com Autofac
#2	Configuração	Isolar ajustes locais de banco
#3	DevOps	Criar workflow GitHub Actions
#4	Arquitetura	Documentar camadas e diagrama
#5	Qualidade	Code smells, SOLID e padrões
#6	Contribuições	Consolidar documentação final
#7	Testes	Adicionar testes de integração
#9	Release	Integrar DEV na main

Contribuição ao projeto original

PR aberto para Matrix-Developers/Car-Rental-App com apenas os três arquivos da Issue #216.



UFOP

Sumário

3 Arquitetura

► Contexto

► Fluxo de Desenvolvimento

► **Arquitetura**

► Refatoração

► Qualidade

► Testes e DevOps

► Resultados



UFOP

Arquitetura Identificada

3 Arquitetura

Camadas

- **WindowsApp:** interface WinForms;
- **Aplicacao:** casos de uso;
- **Dominio:** entidades e contratos;
- **Infra:** banco, logs, PDF;
- **Tests:** testes automatizados.

Leitura arquitetural

A solução segue uma arquitetura em camadas, com separação parcial próxima de Clean Architecture.

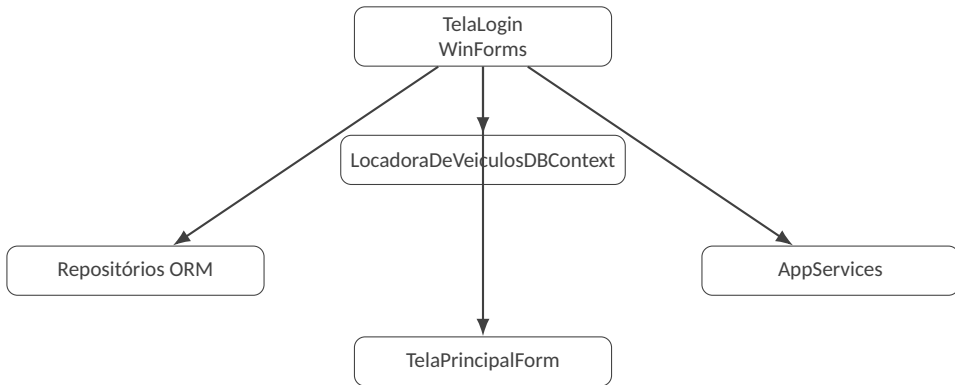
Ponto crítico

A TelaLogin rompia essa separação ao conhecer detalhes da infraestrutura.



Dependências Antes da Refatoração

3 Arquitetura



Problema

A interface estava criando infraestrutura, serviços e a tela principal diretamente.



UFOP

Sumário

4 Refatoração

- ▶ Contexto
- ▶ Fluxo de Desenvolvimento
- ▶ Arquitetura
- ▶ **Refatoração**
- ▶ Qualidade
- ▶ Testes e DevOps
- ▶ Resultados



Antes: Criação Manual de Dependências

4 Refatoração

```
db = new();
servicoAppService = new(new ServicoORM(db));
clienteAppService = new(new ClienteORM(db));
funcionarioAppService = new(new FuncionarioORM(db));
veiculoAppService = new(
    new VeiculoORM(db),
    new ImagemVeiculoORM(db)
);
locacaoAppService = new(new LocacaoORM(db));
```

Diagnóstico

Alto acoplamento entre interface, serviços, repositórios e Entity Framework.



Depois: Dependências pelo Construtor

4 Refatoração

```
private readonly FuncionarioAppService funcionarioAppService;  
private readonly Func<TelaPrincipalForm> telaPrincipalFactory;  
  
public TelaLogin(  
    FuncionarioAppService funcionarioAppService,  
    Func<TelaPrincipalForm> telaPrincipalFactory)  
{  
    InitializeComponent();  
    this.funcionarioAppService = funcionarioAppService;  
    this.telaPrincipalFactory = telaPrincipalFactory;  
}
```

Mudança central

A tela de login recebe apenas o que realmente precisa para autenticar e abrir a próxima tela.



Autofac como Composition Root

4 Refatoração

```
Builder.RegisterType<TelaLogin>()  
    .InstancePerDependency();  
  
Builder.RegisterType<TelaPrincipalForm>()  
    .InstancePerDependency();
```

Resultado

- o container resolve as telas;
- os serviços deixam de ser criados manualmente;
- a tela principal é criada sob demanda;
- a UI perde dependência direta da infraestrutura.



UFOP

Sumário

5 Qualidade

- ▶ Contexto
- ▶ Fluxo de Desenvolvimento
- ▶ Arquitetura
- ▶ Refatoração
- ▶ **Qualidade**
- ▶ Testes e DevOps
- ▶ Resultados



Code Smells Identificados

5 Qualidade

Smell	Problema
Alto acoplamento	TelaLogin criava contexto, repositórios e serviços.
Composition Root incorreto	A tela de login conhecia a árvore de dependências da tela principal.
Código morto	Uma nova tela de login era criada apenas para ser fechada.

Direção da correção

Centralizar composição no Autofac e deixar a tela responsável apenas pelo fluxo de autenticação.



UFOP

Padrões e SOLID

5 Qualidade

Padrões

- **Dependency Injection:** Autofac;
- **Repository:** contratos e ORM;
- **Factory:** `Func<TelaPrincipalForm>`.

Princípios aplicados

- **SRP:** login não compõe infraestrutura;
- **DIP:** UI deixa de depender de concretos;
- redução de acoplamento entre camadas.



UFOP

Qualidade de Código

5 Qualidade

Ferramenta

A análise de qualidade foi registrada com SonarQube for IDE / SonarLint no Visual Studio.

Pontos observados

- criação manual de dependências;
- responsabilidades excessivas;
- código morto;
- APIs obsoletas e bibliotecas legadas.

Escopo

Alertas de bibliotecas legadas foram documentados, mas não alterados por ficarem fora da Issue #216.



UFOP

Sumário

6 Testes e DevOps

- ▶ Contexto
- ▶ Fluxo de Desenvolvimento
- ▶ Arquitetura
- ▶ Refatoração
- ▶ Qualidade
- ▶ **Testes e DevOps**
- ▶ Resultados



UFOP

Testes Automatizados

6 Testes e DevOps

Base existente

- 54 testes unitários;
- 30 testes de aplicação;
- 42 testes ORM;
- total existente: 126.

Novos testes

- projeto `DependencyInjection.Tests`;
- 6 testes de integração;
- validação do Autofac e telas;
- total local: 132 aprovados.



Cobertura dos Novos Testes

6 Testes e DevOps

Teste	Cobertura
Registro da TelaLogin	garante registro no container.
Registro da TelaPrincipalForm	valida a tela principal no Autofac.
Resolução da TelaLogin	cria login com dependências.
Resolução da tela principal	valida árvore principal de serviços.
Factory da tela principal	valida Func<TelaPrincipalForm>.
Instâncias independentes	evita reuso indevido de formulário.



Pipeline CI/CD

6 Testes e DevOps

GitHub Actions

Workflow em `.github/workflows/dotnet-ci.yml`.

Eventos

- push em DEV;
- push em main;
- PR para DEV;
- PR para main.

Etapas

- restore;
- build;
- testes unitários;
- testes de aplicação;
- artefatos de resultado.



Limitações dos Testes

6 Testes e DevOps

Cypress/Selenium

A especificação citava ferramentas voltadas a aplicações web. O projeto é Windows Forms, então Cypress e Selenium não automatizam diretamente a interface desktop.

Decisão adotada

Foram criados testes de integração com MSTest para validar a refatoração real do Autofac.

Infraestrutura local

Testes ORM e parte dos testes de integração exigem SQL Server local. Por isso, ficaram fora do CI e foram validados localmente.



UFOP

Sumário

7 Resultados

- ▶ Contexto
- ▶ Fluxo de Desenvolvimento
- ▶ Arquitetura
- ▶ Refatoração
- ▶ Qualidade
- ▶ Testes e DevOps
- ▶ **Resultados**



UFOP

Resultados da Entrega

7 Resultados

Técnico

- Issue #216 resolvida;
- dependências centralizadas no Autofac;
- menor acoplamento da TelaLogin;
- PR aberto para o projeto original.

Processo

- documentação em /documentacao;
- PRs pequenos e organizados;
- revisão entre membros;
- CI com build e testes independentes.



UFOP

Conclusão

7 Resultados

Síntese

O trabalho aplicou manutenção corretiva/evolutiva, refatoração, análise arquitetural, testes e DevOps em um projeto real.

Principal ganho

A TelaLogin deixou de compor manualmente dependências e passou a utilizar o fluxo de injeção já previsto pelo projeto.

Limitação assumida

A automação da interface desktop não foi feita com Cypress/Selenium; a validação foi feita com testes de integração e documentação explícita da limitação.



UFOP

Obrigado!